

VIETNAM NATIONAL UNIVERSITY - HCMC
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FALCUTY OF COMPUTER SCIENCE AND ENGINEERING



LOGIC DESIGN PROJECT (CO3091)

Project Report on:

Low Power AES-256 Crypto Accelerator

Class CC06 - Semester 251

Course Instructor: Phạm Kiều Nhật Anh

Student:	Ngô Nguyễn Thành Nhân	2352849
	Trần Thanh Huy	2352414
	Trần Hoàng Thiên Phú	2352923

December 8th 2025

Table of Contents

I. Introduction	1
1.1 Problem Statement	1
1.2 Project Objectives	1
1.3 Scope and Limitations	2
1.3.1 Project Scope	2
1.3.2 Limitations	2
II. Theoretical Background	3
2.1 Advanced Encryption Standard (AES)	3
2.1.1 Overview of Symmetric Cryptography	3
2.1.2 The AES-256 Algorithm Structure	3
2.1.3 AddRoundKey Operation	3
2.1.4 Round-by-Round Execution (AES-256)	4
2.1.5 Key Expansion and Round Key Generation	5
2.1.6 SubBytes Transformation	5
2.1.7 ShiftRows Transformation	6
2.1.8 MixColumns Transformation	7
III. Implementation	7
3.1 Implement AES	7
3.1.1 Design core module AES encrypt and AddRoundKey module	7
3.1.2 Implement SubBytes module	12
3.1.3 Implement ShiftRows module	13
3.1.4 Implement MixColumns module	15
3.1.5 Implement keyExpansion module	17
3.2 Implement UART	20
3.2.1 Hardware design	20
3.2.2 Software design	21
3.3 Implement OpenLane	25
3.3.1 Design Setup and Directory Structure	25
3.3.2 The RTL-to-GDSII Flow Execution	25
IV. Results and Discussion	26
4.1 Functional Verification	26
4.2 Initialization and Input Loading	26
4.2.1 Iterative Round Execution	27
4.2.2 Final Output Verification	28
4.3 Performance Evaluation	28
4.3.1 Latency Analysis	28

4.3.2 Throughput Calculation	28
4.4 ASIC Implementation Results(OpenLane)	29
4.4.1 Resource Utilization and Area	29
4.4.2 Power Analysis	30
4.4.3 Efficiency Metric: Throughput per Watt (TPW)	30
4.4.4 GDSII Layout Generation	31
4.5 FPGA Prototyping Results (Vivado)	33
4.5.1 Area Efficiency	33
4.5.2 Timming Analysis	34
4.5.3 On-chip Power Analysis	35
4.6 Comparative Analysis: FPGA vs. ASIC Timing	35
4.6.1 Discussion on Technology Node Differences	36
4.5.4 Hardware Execution Verification	36

V. Conclusion	37
----------------------	-----------

I. Introduction

1.1 Problem Statement

Modern embedded and Internet of Things (IoT) devices require robust data security, often relying on the Advanced Encryption Standard (AES-256). Implementing this algorithm purely in software on a constrained processor, such as a single-core RISC-V or ARM unit, incurs significant **latency overhead** and results in **high power consumption** per encrypted bit. This presents a critical design conflict between security needs and system efficiency.

The central problem addressed by this project is the lack of a highly **energy-efficient** and **high-throughput** solution for AES-256 encryption within resource-limited systems. This solution must not only offload the intensive cryptographic task from the general-purpose processor but must also be validated through a complete hardware design flow utilizing a manufacturable open-source technology node, specifically the **SkyWater 130nm (SKY130) PDK**. Furthermore, the final integrated system must be demonstrably functional, capable of accepting user input and outputting verified ciphertext via a prototype platform (Arty Z7 FPGA).

1.2 Project Objectives

The primary goal of this project is to design, implement, and validate a highly efficient hardware-software co-design system for AES-256 encryption. The measurable objectives are:

1. **Hardware Acceleration Design:** Implement the full 14-round AES-256 encryption algorithm in synthesizable Verilog, utilizing an **iterative (area-optimized)** structure to minimize resource utilization while operating as a multi-cycle accelerator.
2. **Processor Integration:** Successfully integrate the custom AES Accelerator core (AES_0) with the **Zynq-7000 Processing System (PS)**, using the AXI-Lite bus protocol for memory-mapped data transfer and control.
3. **Low-Power ASIC Validation:** Execute a complete Physical Design flow using **OpenLane** and the **SkyWater 130nm (SKY130) PDK**. Analyze the resulting Power, Performance, and Area (PPA) figures, specifically optimizing the design to achieve maximum **Throughput per Watt (TPW)** at the 130nm node.
4. **System Prototyping and Demonstration:** Implement the entire system onto the **Arty Z7 FPGA board** using **Vitis** for software development, accepting message input and successfully displaying the final ciphertext via the terminal.

1.3 Scope and Limitations

The project scope and the subsequent analysis detailed in this report are confined by specific design boundaries and practical constraints.

1.3.1 Project Scope

The following key elements were successfully implemented and are within the scope of this work:

- **Functionality:** Full implementation and verification of the AES-256 encryption algorithm.
- **Design Style:** Utilization of an **iterative architecture** for the AES core, prioritizing minimum area and resource utilization.
- **System Integration:** Successful interfacing of the custom AES accelerator (AES_0) with the **Zynq-7000 Processing System (PS)** via AXI-Lite for hardware-software communication.
- **Physical Validation:** Generation of full Power, Performance, and Area (PPA) figures using the **OpenLane flow and the SkyWater 130nm (SKY130) PDK**, providing manufacturable physical design metrics.
- **Demonstration:** Prototyping the integrated system on the Arty Z7 board, including plaintext input and ciphertext output to the terminal.

1.3.2 Limitations

The following factors were deemed out of scope or represent unavoidable constraints:

- **Cryptographic Modes:** The implementation is restricted to the basic Electronic Codebook (ECB) mode of operation for simplicity and proof-of-concept; secure operational modes like CBC or GCM are not included.
- **Decryption Functionality:** The accelerator is designed exclusively for the AES-256 **encryption block**. The decryption logic is not implemented.
- **Physical Security:** No specific hardware countermeasures were implemented to mitigate known physical security threats, such as Side-Channel Analysis (SCA) or timing attacks.
- **ASIC Fabrication:** While the design utilizes the manufacturable SKY130 PDK and generates a valid GDSII layout, the actual tape-out and physical fabrication of the chip were not performed due to the project timeline.
- **RISC-V Processor:** Due to time and complexity constraints for the FPGA demonstration, the Zynq PS ARM core was utilized for running embedded software, rather than implementing a full soft-core RISC-V processor on the Programmable Logic (PL) fabric.

II. Theoretical Background

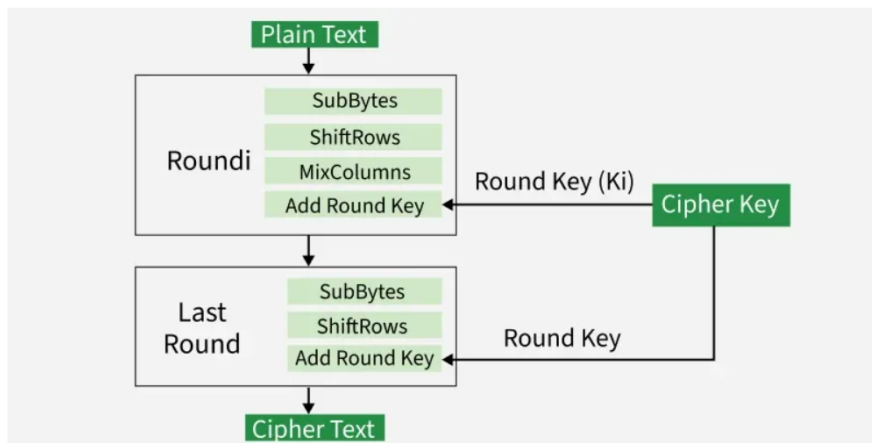
2.1 Advanced Encryption Standard (AES)

2.1.1 Overview of Symmetric Cryptography

The Advanced Encryption Standard (AES) is a modern, widely adopted symmetric block cipher. As a symmetric cipher, it uses the same key for both encryption and decryption, distinguishing it from public-key (asymmetric) algorithms. The "block" nature of AES means it operates on fixed-size data blocks, specifically 128 bits, regardless of the key length.

2.1.2 The AES-256 Algorithm Structure

The AES-256 algorithm operates on 128-bit blocks of data, which are represented internally as a 4×4 matrix of bytes (16 bytes) in a column-major arrangement. The encryption process consists of an initial AddRoundKey step, followed by 13 standard rounds, and concludes with a final round. Each standard round comprises four distinct transformations: SubBytes, ShiftRows, MixColumns, and AddRoundKey. The Key Expansion module is responsible for generating the 14 unique 128-bit sub-keys required for these rounds.



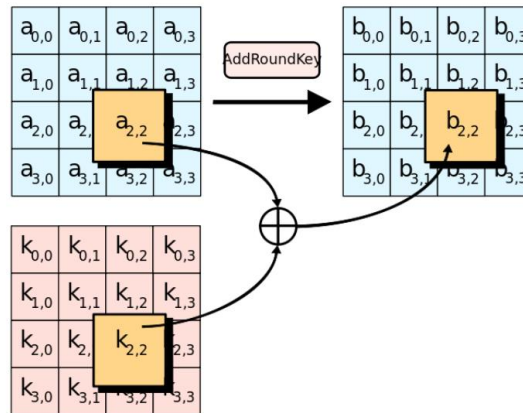
Ex. 1: The full 14-Round AES-256 Encryption Flow.

2.1.3 AddRoundKey Operation

The AddRoundKey operation is the simplest and final step of a standard round. It combines the output of the preceding transformations with a 128-bit sub-key derived from the Key Expansion routine. The operation is performed using a simple bitwise XOR ($\oplus$) between the 16 bytes of the state matrix and the 16 bytes of the round key. This step introduces key material into the encryption process and is performed in all rounds, including the initial step and the final round.

AES: Add Round Key

- XOR state with 128-bits of the round key



31

Ex. 2: AES AddRoundKey XOR operation

2.1.4 Round-by-Round Execution (AES-256)

The AES-256 algorithm is performed over a total of 14 rounds to ensure maximum cryptographic strength. For a hardware accelerator utilizing an iterative, multi-cycle design, understanding the operations of each round type is critical for designing the controlling Finite State Machine (FSM). The 14 rounds are categorized into three execution phases: the Initial Round, the Middle Rounds, and the Final Round.

1. **Initial Round:** Only a single transformation is executed. The 128-bit plaintext is XORed with the first 128-bit sub-key (K_0).

- **Operation:** AddRoundKey

2. **Middle Rounds (Rounds 1 to 13):** The encryption process iterates through 13 identical rounds, each executing the full sequence of four core transformations:

- **Sequence:** SubBytes $\rightarrow$ ShiftRows $\rightarrow$ MixColumns $\rightarrow$ AddRoundKey

This repeated application of substitution (SubBytes) and permutation and mixing (ShiftRows, MixColumns) ensures robust confusion and diffusion.

3. **Final Round (Round 14):** The last round is structurally different to maintain the invertibility of the cipher, which is necessary for decryption. The **MixColumns** transformation is deliberately **omitted**.

- **Sequence:** SubBytes $\rightarrow$ ShiftRows $\rightarrow$ AddRoundKey

After the final AddRoundKey operation, the resulting 128-bit state is output as the ciphertext.

2.1.5 Key Expansion and Round Key Generation

The AES-256 algorithm requires a separate 128-bit key for each round of the encryption process. Since AES-256 consists of 14 rounds plus one initial key addition, the Key Expansion unit must generate a total of 15 Round Keys (K_0 through K_{14}) from the original 256-bit Cipher Key.

The Key Expansion process expands the 256-bit (32-byte) master key into a Key Schedule of 240 bytes (1,920 bits). This schedule is generated word-by-word (where 1 word = 32 bits), producing a total of 60 words, labeled w_0 to w_{59} .

The expansion is generated according to the following rules:

- **Initial Words (w_0 to w_7):** The first 8 words are simply the bytes of the original 256-bit master key.
- **Subsequent Words:** The remaining words (w_8 to w_{59}) are derived recursively. The algorithm depends on the position of the word index i :
 1. **If i is a multiple of 8:** The generation involves a complex non-linear transformation:

$$w_i = w_{i-8} \oplus \text{SubWord}(\text{RotWord}(w_{i-1})) \oplus \text{Rcon}_{i/8}$$

2. **If $i+4$ is a multiple of 8:** (Specific to AES-256) An additional SubWord step is applied to add non-linearity to the long key:

$$w_i = w_{i-8} \oplus \text{SubWord}(w_{i-1})$$

3. **Otherwise:** The word is generated by a simple XOR of previous words:

$$w_i = w_{i-8} \oplus w_{i-1}$$

The three primitive functions used in this process are:

- **RotWord:** Performs a cyclic left shift on a 32-bit word $[a_0, a_1, a_2, a_3]$ to produce $[a_1, a_2, a_3, a_0]$.
- **SubWord:** Applies the standard AES S-box substitution to each of the four bytes in the word independently.
- **Rcon (Round Constant):** A 32-bit constant where the first byte is a power of 2 in the Galois Field $GF(2^8)$, and the remaining three bytes are zero. This ensures that the keys for each round are mathematically distinct, preventing symmetry attacks.

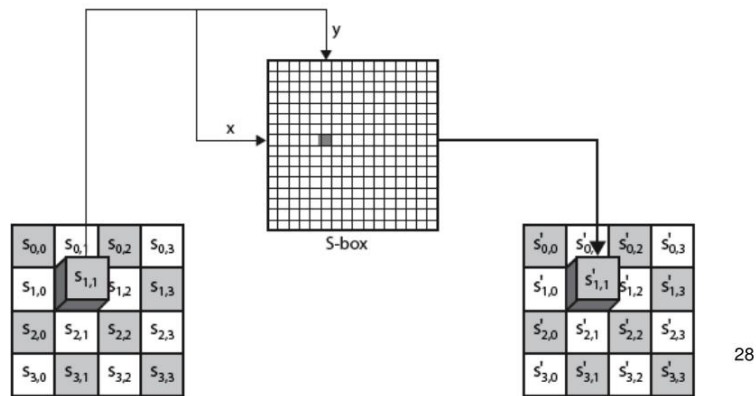
2.1.6 SubBytes Transformation

The SubBytes step implements the substitution property of the cipher. In this non-linear step, every byte in the 4×4 state matrix is independently substituted

by a corresponding output byte from a fixed lookup table, known as the **S-box**. The substitution is specifically designed to ensure a byte is never replaced by itself, nor is it substituted by its complement. The result remains a 4×4 matrix of bytes.

AES: SubBytes (S-box)

- a simple substitution of each byte
- a byte = two nibbles
- S-box has 16x16 entries: all possible 8-bit values
- each byte of state is replaced by byte indexed by row (left 4-bits) & column (right 4-bits)



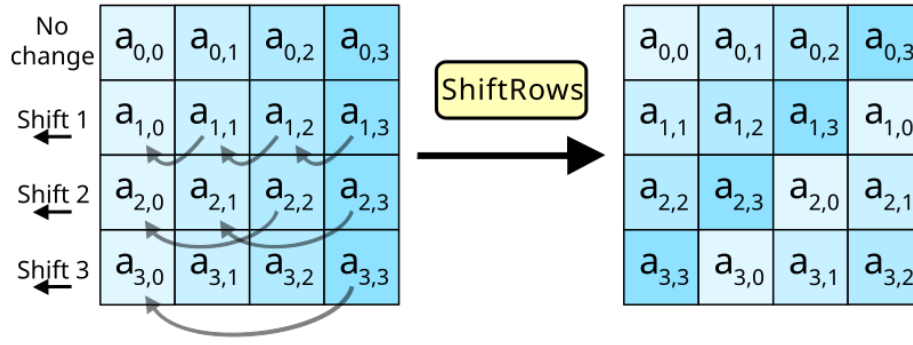
Ex. 3: AES S-Box substitution diagram

2.1.7 ShiftRows Transformation

The ShiftRows step implements the primary permutation of the cipher, providing diffusion across the columns. This step involves a left circular shift of the bytes in each of the four rows by a fixed number of positions:

- **Row 1:** Remains unshifted (0 shifts).
- **Row 2:** Is shifted one position to the left.
- **Row 3:** Is shifted two positions to the left.
- **Row 4:** Is shifted three positions to the left.

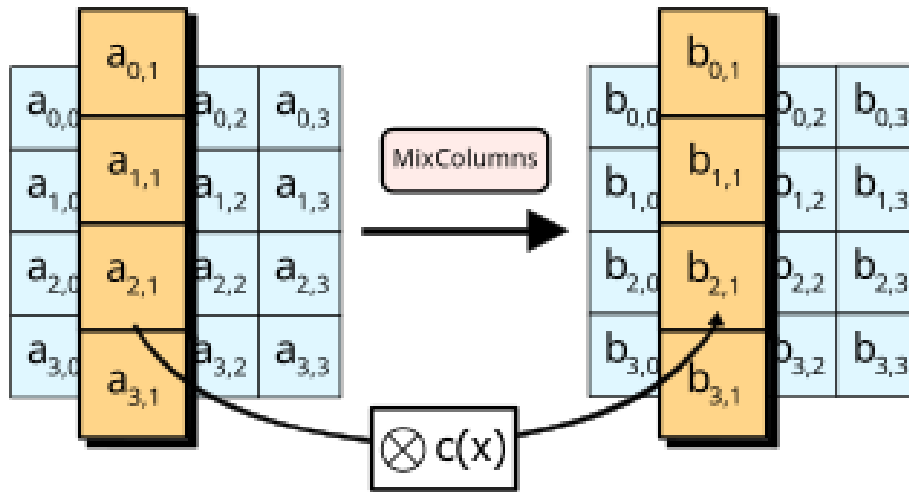
This rearrangement mixes data from different columns, which is crucial for achieving strong diffusion.



Ex. 4: AES ShiftRows operation visualization

2.1.8 MixColumns Transformation

The MixColumns step is a linear mixing operation achieved through matrix multiplication. Each column of the 4×4 state matrix is treated as a 4-byte vector and multiplied by a fixed, pre-defined 4×4 matrix. This operation ensures that each byte in the resultant column depends on all four bytes of the original column, providing strong diffusion across the entire state. As dictated by the AES standard, the MixColumns transformation is skipped entirely in the last round of encryption.



Ex. 5: AES MixColumns matrix multiplication diagram

III. Implementation

3.1 Implement AES

3.1.1 Design core module AES encrypt and AddRoundKey module

a) Overview

The core control logic of the AES_Encrypt module is implemented using a Finite State Machine (FSM). This FSM orchestrates the encryption sequence, ensuring that the 128-bit data block undergoes the correct transformations (SubBytes, ShiftRows, MixColumns) and the AddRoundKey operation for exactly 14 rounds (as required for AES-256).

b) Signal Interface

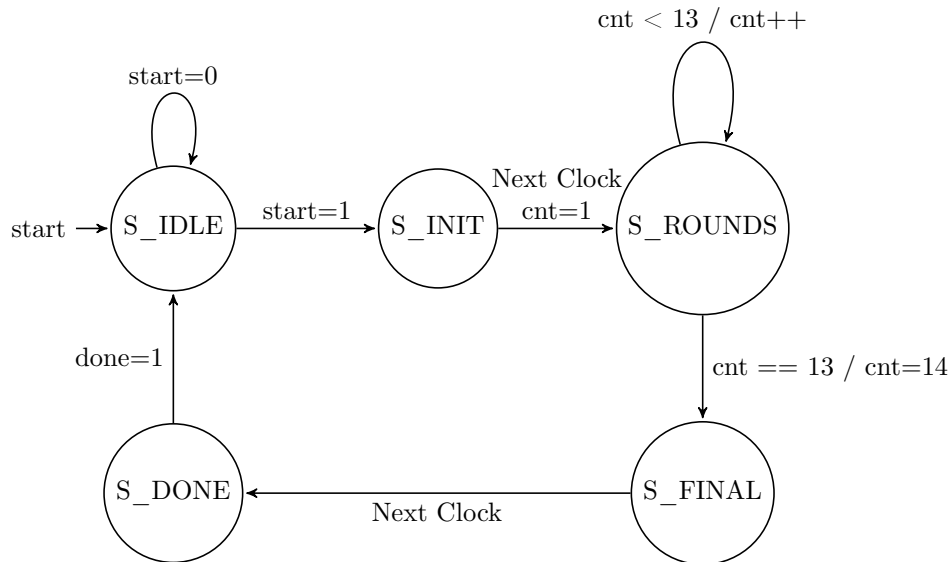
The input and output signals of the module are described in the table below:

Table 1: Signal Interface Description

Signal Name	Width	Direction	Description
clk	1 bit	Input	System clock synchronization signal
reset	1 bit	Input	System reset signal (Active High)
start	1 bit	Input	Trigger pulse to start encryption process
in	128 bits	Input	Input data block (Plaintext)
key	256 bits	Input	Secret key used for AES-256
out	128 bits	Output	Encrypted data block (Ciphertext)
done	1 bit	Output	Flag indicating encryption completion

c) Implement FSM and AddRoundKey

The operation of the AES-256 Core is governed by the Finite State Machine. The FSM coordinates the transition between encryption stages based on the system clock and the internal round counter.



Ex. 6: FSM of module AES_Encrypt

The functionality of each state is described below:

- **S_IDLE (Reset/Wait):** This is the default state upon system reset. The FSM remains in this state, monitoring the **start** input signal. When **start** is asserted, the FSM transitions to the initialization state.
- **S_INIT_ADD_KEY (Round 0):** Performs the initial AddRoundKey operation, XORing the plaintext with the initial key. The round counter (**cnt**) is set to 1, then transitions to the main loop on the next clock cycle.
- **S_ROUNDS (Main Loop):** Executes SubBytes, ShiftRows, MixColumns, and AddRoundKey for rounds 1–13.
 - If **cnt < 13**: remain in this state and increment counter.
 - When **cnt = 13**: transition to S_FINAL_ROUND and set counter to 14.

- **S_FINAL_ROUND (Round 14):** Executes the final round without MixColumns, as required by the AES specification.
- **S_DONE (Completion):** The final state. The `done` flag is asserted to indicate valid output. The FSM returns to `S_IDLE` in the next clock cycle.

The design uses a synchronous `always` block triggered by the rising edge of the clock (`posedge clk`). This block is responsible for:

1. **Reset:** Asynchronously resetting the system to the `IDLE` state.
2. **State Transition:** Moving the FSM to the next state (`next_state`).
3. **Data Update:** Updating the encryption result (`state_data_reg`) and the round counter (`round_counter_reg`).
4. **Output Logic:** driving the final out data and the done flag when the process completes.

Listing 1: FSM and AddRoundKey

```

1  always @(posedge clk or posedge reset) begin
2      if (reset) begin
3          state_reg          <= S_IDLE;
4          round_counter_reg  <= 0;
5          state_data_reg     <= 128'b0;
6          out                <= 128'b0;
7          done               <= 1'b0;
8      end else begin
9          state_reg          <= next_state;
10         round_counter_reg  <= next_round_counter;
11         state_data_reg     <= next_state_data;
12
13         if (reset) begin
14             done <= 1'b0;
15         end else if (state_reg == S_DONE) begin
16             out <= next_state_data;
17             done <= 1'b1;
18         end else if (start) begin
19             done <= 1'b0;
20         end
21     end
22 end

```

The combinational logic block (`always @(*)`) determines the next state and the next data value based on the current state. This is where the specific operations for each round, including `AddRoundKey`, are integrated.

The FSM operates through the following key states:

- **A. Initial AddRoundKey (Round 0)**

In the `S_INIT_ADD_KEY` state, the AES algorithm requires the plaintext to be XORed directly with the initial key.

- **Implementation:** The code executes `next_state_data = in ■ round_key;`.
- **Note:** The operator `■` represents the XOR operation, which **IS** the `AddRoundKey` step.

• B. Main Rounds (Rounds 1–13)

In the `S_ROUNDS` state, the FSM captures the result of the full transformation loop.

- **Implementation:** `next_state_data = round_result;`
- The signal `round_result` (assigned outside this block) contains the data after passing through *SubBytes*, *ShiftRows*, *MixColumns*, and finally **AddRoundKey** (XOR).
- **Loop Control:** The `round_counter_reg` increments until it reaches 13, at which point the FSM transitions to the final round.

• C. Final Round (Round 14)

In the `S_FINAL_ROUND` state, the *MixColumns* step is skipped, but the **AddRoundKey** is still performed at the end. The result is captured and prepared for output.

Listing 2: FSM and AddRoundKey

```

1  always @(*) begin
2      next_state = state_reg;
3      next_round_counter = round_counter_reg;
4      next_state_data = state_data_reg;
5
6      case (state_reg)
7          S_IDLE: begin
8              if (start) begin
9                  next_state = S_INIT_ADD_KEY;
10                 next_round_counter = 0;
11             end
12         end
13
14         S_INIT_ADD_KEY: begin
15             next_state_data = in ^ round_key;
16
17             next_round_counter = 1;
18             next_state = S_ROUNDS;
19         end
20
21         S_ROUNDS: begin
22             next_state_data = round_result;
23
24             if (round_counter_reg < 13) begin
25                 next_round_counter = round_counter_reg + 1;
26                 next_state = S_ROUNDS;
27             end else begin
28                 next_round_counter = 14;
29                 next_state = S_FINAL_ROUND;
30             end
31         end
32
33         S_FINAL_ROUND: begin
34             next_state_data = round_result;
35             next_state = S_DONE;
36         end
37
38         S_DONE: begin
39             next_state = S_IDLE;
40         end
41
42         default: begin
43             next_state = S_IDLE;
44         end
45     endcase
46 end

```

d) Data Path Architecture

The data path interconnects functional modules to perform the mathematical transformations required by the AES-256 algorithm. This section describes the key components and control logic that implement the AES data flow.

The top-level module instantiates the following sub-components:

- **keyExpansion:** Generates round keys from the original 256-bit secret key based on the current round number. This module implements the AES key schedule algorithm to produce 15 distinct round keys (including the initial key).
- **subBytes:** Performs the non-linear byte substitution using the AES S-Box. Each byte of the state matrix is replaced according to a fixed lookup table that provides cryptographic confusion.
- **shiftRows:** Executes the cyclic shift operation on each row of the state matrix. This diffusion layer ensures that bytes are permuted across different columns.
- **mixColumns:** Performs matrix multiplication in the Galois Field $GF(2^8)$. This transformation operates on each column of the state, mixing the data to provide additional diffusion across the cipher.

A critical aspect of the design is the handling of differences between regular rounds and the final round. The AES specification requires that the final round (round 14) omits the MixColumns transformation. To implement this efficiently in hardware, a multiplexer controls the data path as follows:

Listing 3: Multiplexer for final round handling

```

1 // If it's the final round, take data from ShiftRows (skip MixColumns)
2 // If it's a regular round, take data from MixColumns
3 assign ark_in = (state_reg == S_FINAL_ROUND) ? sr_out : mc_out;
4
5 // Perform AddRoundKey (XOR with round key)
6 assign round_result = ark_in ^ round_key;
```

This multiplexing logic ensures hardware compliance with the AES standard while maintaining efficient resource utilization. The control signal (`state_reg == S_FINAL_ROUND`) is generated by the FSM controller described.

The data flow proceeds as follows for each round:

1. **Initial Round:** $\text{state_data} \oplus \text{round_key}[0]$
2. **Rounds 1-13:** SubBytes $\rightarrow$ ShiftRows $\rightarrow$ MixColumns $\rightarrow$ AddRoundKey
3. **Final Round (14):** SubBytes $\rightarrow$ ShiftRows $\rightarrow$ AddRoundKey (skip MixColumns)

The multiplexer shown in Code effectively implements the conditional execution of MixColumns, ensuring the final round follows the AES specification exactly.

This approach minimizes hardware complexity while maintaining full compliance with the algorithm requirements.

3.1.2 Implement SubBytes module

The SubBytes transformation is a fundamental non-linear operation in the AES algorithm that provides cryptographic confusion by substituting each byte of the state matrix according to a predefined S-Box lookup table.

The SubBytes module implements a parallel 16-byte substitution using a generated array of individual S-Box units. This parallel architecture ensures single-cycle operation for the entire 128-bit state.

The module is implemented in Verilog as shown in Code:

Listing 4: SubBytes module implementation

```

1  `timescale 1ns / 1ps
2
3  module subBytes(
4      input wire [127:0] in,
5      output wire [127:0] out
6  );
7
8      genvar i;
9      generate
10         for (i = 0; i < 128; i = i + 8) begin : sbbox_loop
11             sbbox sbbox_inst (
12                 .a(in[i+:8]),
13                 .c(out[i+:8])
14             );
15         end
16     endgenerate
17
18 endmodule

```

- **Parallel Processing:** The module processes all 16 bytes (128 bits) simultaneously using a `generate` loop that instantiates 16 identical S-Box units.
- **Byte-wise Operation:** Each S-Box unit operates on an 8-bit input and produces an 8-bit output, corresponding to one byte of the AES state matrix.
- **Slice Operator:** The Verilog slice operator `[i+:8]` efficiently extracts contiguous 8-bit segments from the 128-bit input bus, where `i` increments by 8 for each iteration (0, 8, 16, ..., 120).
- **Timing:** The module operates combinatorially with zero clock cycles latency, as all S-Box units process their inputs simultaneously.

The individual S-Box module implements the AES substitution box using a combinational `case` statement that directly maps each 8-bit input to its corresponding 8-bit output according to the AES standard. This approach provides an efficient and readable implementation optimized for FPGA synthesis.

The S-Box module shown in Code uses a comprehensive `case` statement to implement the complete 256-byte substitution:

Listing 5: S-Box lookup table implementation

```

1 module sbbox(
2     input wire [7:0] a,
3     output reg [7:0] c
4 );
5 always @(*) begin
6     case (a)
7         8'h00: c = 8'h63;    8'h01: c = 8'h7c;
8         8'h02: c = 8'h77;    8'h03: c = 8'h7b;
9         8'h04: c = 8'hf2;    8'h05: c = 8'h6b;
10        8'h06: c = 8'h6f;    8'h07: c = 8'h5c;
11        8'h08: c = 8'h3b;    8'h09: c = 8'h1b;
12        8'h10: c = 8'h4b;    8'h11: c = 8'h8c;
13        8'h12: c = 8'h9c;    8'h13: c = 8'h7d;
14        8'h14: c = 8'hfa;    8'h15: c = 8'h59;
15        8'h16: c = 8'h47;    8'h17: c = 8'hfb;
16        // ... (complete 256 entries)
17        8'hfe: c = 8'h75;    8'hff: c = 8'h9f;
18    endcase
19 end
20 endmodule

```

- **Combinational Logic:** The `always @(*)` block ensures the module operates as pure combinational logic with no clock dependencies.
- **Complete Coverage:** The `case` statement includes all 256 possible input values (8'h00 to 8'hff), ensuring no undefined outputs.
- **Synthesis Optimization:** Modern FPGA synthesis tools typically optimize the `case` statement into an efficient lookup table using LUT (Look-Up Table) resources, often implementing it as a distributed ROM.
- **No Default Case:** The absence of a `default` case is intentional, as all 256 input possibilities are explicitly covered, preventing latch inference during synthesis.

Efficient LUT utilization on Artix-7, maximum speed (>300 MHz), constant-time execution for side-channel resistance, and simplified verification against AES standard.

Large code footprint, no runtime configurability, higher power consumption, and potential synthesis warnings about case statement size.

3.1.3 Implement ShiftRows module

The ShiftRows module implements the AES row shifting transformation through efficient wire reordering logic. This operation permutes bytes within the state matrix by cyclically shifting each row according to the AES specification. The hardware implementation is purely combinational and is written using assign statements. There is no control logic and no clocking, because ShiftRows is simply a fixed byte permutation.

The module implements the AES ShiftRows transformation according to the following mapping pattern, where the 16-byte state is arranged as a 4×4 matrix:

$$\text{State Matrix: } \begin{bmatrix} a_{00} & a_{01} & a_{02} & a_{03} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \\ a_{30} & a_{31} & a_{32} & a_{33} \end{bmatrix} \xrightarrow{\text{ShiftRows}} \begin{bmatrix} a_{00} & a_{01} & a_{02} & a_{03} \\ a_{11} & a_{12} & a_{13} & a_{10} \\ a_{22} & a_{23} & a_{20} & a_{21} \\ a_{33} & a_{30} & a_{31} & a_{32} \end{bmatrix}$$

The 128-bit input is divided into 16 bytes following the AES column-major indexing:

Listing 6: ShiftRows module implementation

```

1 wire [7:0] r0c0 = in[127:120];
2   wire [7:0] r1c0 = in[119:112];
3   wire [7:0] r2c0 = in[111:104];
4   wire [7:0] r3c0 = in[103:96];
5
6   wire [7:0] r0c1 = in[95:88];
7   wire [7:0] r1c1 = in[87:80];
8   wire [7:0] r2c1 = in[79:72];
9   wire [7:0] r3c1 = in[71:64];
10
11  wire [7:0] r0c2 = in[63:56];
12  wire [7:0] r1c2 = in[55:48];
13  wire [7:0] r2c2 = in[47:40];
14  wire [7:0] r3c2 = in[39:32];
15
16  wire [7:0] r0c3 = in[31:24];
17  wire [7:0] r1c3 = in[23:16];
18  wire [7:0] r2c3 = in[15:8];
19  wire [7:0] r3c3 = in[7:0];
20 endmodule

```

This step maps each byte of the input state into a symbolic position (row, column) for clarity.

For example:

- r0c0 represents state by a_{00} .
- r1c2 represents a_{12} .
- r3c3 represents a_{33} .

Each row is reassigned according to the AES rotation offset.

Listing 7: ShiftRows module implementation

```

1  \\shift = 0
2  assign shifted[127:120] = r0c0;
3  assign shifted[119:112] = r1c1;
4  assign shifted[111:104] = r2c2;
5  assign shifted[103:96] = r3c3;
6
7  \\shift = 1
8  assign shifted[95:88] = r0c1;
9  assign shifted[87:80] = r1c2;
10 assign shifted[79:72] = r2c3;
11 assign shifted[71:64] = r3c0;
12 endmodule
13
14 \\shift = 2
15 assign shifted[63:56] = r0c2;
16 assign shifted[55:48] = r1c3;
17 assign shifted[47:40] = r2c0;
18 assign shifted[39:32] = r3c1;
19
20 \\shift = 3
21 assign shifted[31:24] = r0c3;
22 assign shifted[23:16] = r1c0;
23 assign shifted[15:8] = r2c1;
24 assign shifted[7:0] = r3c2;

```

Each assignment corresponds exactly to the AES theoretical mapping.

For example:

- Output byte (1,0) receives input byte (1,1) due to the 1-byte left shift of row 1.

- Output byte (3,1) receives input byte (3,0) due to the 3-byte rotation of row 3.

This module is implemented using purely combinational logic, without any registers or clocked elements. All assign statements directly connect input and output nets, allowing the synthesis tool to translate the slicing operations into fixed routing paths instead of LUT-based logic. As a result, the design operates in constant time with no data-dependent branching, making it appropriate for cryptographic hardware. Furthermore, because the transformation is realized entirely through wiring, it consumes virtually no logic resources.

3.1.4 Implement MixColumns module

The mixColumns module provides a fully combinational implementation of the AES MixColumns transformation. In AES-128, the state is represented as a 4×4 matrix of bytes, and MixColumns operates independently on each of the four columns. This hardware module receives the 128-bit state as a flat vector (`state_in`) and outputs the transformed state (`state_out`) after applying the MixColumns step.

The module begins by defining two helper functions, `mb2` and `mb3`, which implement finite-field multiplications required in AES:

- multiplication by 2 in $\text{GF}(2^8)$.
- multiplication by 3 in $\text{GF}(2^8)$.

These operations are then reused inside a generate loop that processes all four columns of the AES state.

The entire module is purely combinational: there are no registers, flip-flops, counters, or clock dependencies. All outputs are directly computed from inputs through combinational logic.

a) Multiply by 2:

Listing 8: MixColumns module implementation

```

1 function [7:0] mb2;
2   input [7:0] x;
3   begin
4     mb2 = (x[7] == 1) ? ((x << 1) ^ 8'h1b) : (x << 1);
5   end
6 endfunction

```

This function implements multiplication by 2 in $\text{GF}(2^8)$. In hardware, the left shift is simply rewiring the bits, and the conditional XOR with 0x1B performs the AES polynomial reduction. Synthesis tools map the ternary operator into a 2-to-1 multiplexer plus one 8-bit XOR block. No registers are created; the function is purely combinational logic and inlined within the datapath.

b) Multiply by 3:

Listing 9: MixColumns module implementation

```

1 function [7:0] mb3;
2   input [7:0] x;
3   begin
4     mb3 = mb2(x) ^ x;
5   end
6 endfunction

```

This uses the definition

$$3 \cdot x = (2 \cdot x) \oplus x$$

The synthesizer expands `mb2(x)` into the same combinational network described above, then adds one XOR layer. As a result, `mb3` consists of the logic of `mb2` plus an 8-bit XOR, without any duplication of multipliers.

c) Column processing using a generate-for loop:

Listing 10: MixColumns module implementation

```

1 genvar i;
2 generate
3   for (i = 0; i < 4; i = i + 1) begin : m_col

```

The generate construct executes at compile time. The tool unrolls the loop into four independent hardware blocks, each operating on one AES column (32 bits). Bit slicing `(i*32 + offset) +: 8` selects the individual bytes of each column through static wiring. No runtime iteration exists, it is full hardware duplication, matching AES's 4-column structure.

d) MixColumns transformation: Each output byte is computed by multiplying the column vector with the fixed AES MixColumns matrix. The module uses only:

- `mb2`
- `mb3`
- direct wires
- XOR networks

This forms a purely combinational MixColumns implementation.

e) Output byte 3:

Listing 11: MixColumns module implementation

```

1 assign state_out[(i*32+24) +: 8] =
2   mb2(state_in[(i*32+24) +: 8]) ^
3   mb3(state_in[(i*32+16) +: 8]) ^
4   state_in[(i*32+8) +: 8]
5   state_in[i*32 +: 8];

```

Implements

$$out_3 = 02 \cdot a_3 \oplus 03 \cdot a_2 \oplus a_1 \oplus a_0.$$

The hardware consists of the `mb2` network, the `mb3` network, and a multi-input XOR tree.

f) Output byte 2:

Listing 12: MixColumns module implementation

```
1 assign state_out[(i*32+16) +: 8] =
2     state_in[(i*32+24) +: 8] ^
3     mb2(state_in[(i*32+16) +: 8]) ^
4     mb3(state_in[(i*32+8) +: 8]) ^
5     state_in[i*32 +: 8];
```

Implements

$$\text{out}_2 = a_3 \oplus (02 \cdot a_2) \oplus (03 \cdot a_1) \oplus a_0.$$

g) Output byte 1:

Listing 13: MixColumns module implementation

```
1 assign state_out[(i*32+8) +: 8] =
2     state_in[(i*32+24) +: 8] ^
3     state_in[(i*32+16) +: 8] ^
4     mb2(state_in[(i*32+8) +: 8]) ^
5     mb3(state_in[i*32 +: 8]);
```

Implements

$$\text{out}_1 = a_3 \oplus a_2 \oplus (02 \cdot a_1) \oplus a_0.$$

The entire module is implemented as pure combinational logic, allowing MixColumns to complete in a single cycle. It does not use LUT RAM or hardware multipliers; instead, it relies only on XOR gates, simple multiplexers, and wiring. The design is constant-time with no data-dependent branches. During synthesis, the generate-loop is fully unrolled, producing four identical MixColumns datapaths for each AES column.

3.1.5 Implement keyExpansion module

The keyExpansion module is responsible for generating the Round Keys required for each iteration of the AES encryption process. Since the system implements AES-256, the 256-bit Master Key must be expanded into a larger key schedule.

a) Internal Registers and State Management

The module maintains a 256-bit register, `key_buffer`, which acts as a sliding window to store the current key context.

- **Initialization:** At the beginning of the operation (`round_num == 0`), the Master Key is loaded into this buffer.
- **Word Extraction:** The logic extracts the most significant word (`old_word`) and the least significant word (`prev_word`) to perform the expansion algorithm.

Listing 14: keyExpansion module implementation

```
1 reg [255:0] key_buffer;
2 reg [7:0] rcon;
3 wire [31:0] prev_word;
4 wire [31:0] old_word;
5 wire [31:0] rot_prev_word;
6
7 assign prev_word = key_buffer[31:0];
8 assign old_word = key_buffer[255:224];
9 assign rot_prev_word = {prev_word[23:0], prev_word[31:24]};
```


b) The Transformation (RotWord, SubWord, Rcon)

The AES Key Schedule core involves complex transformations applied to specific words. For AES-256, the transformation rule depends on the position of the word within the key block.

1. **Condition Logic:** The signal `is_complex_step` determines the type of transformation required.
 - If `is_complex_step` is high (Start of a 256-bit block), the algorithm applies RotWord + SubWord + Rcon.
 - Otherwise (Middle of the block), it may apply SubWord only (specific to AES-256) or pass the value through.
2. **S-Box Instantiation:** The module instantiates 4 parallel S-boxes using a generate loop to perform the SubWord operation efficiently in a single clock cycle.

Listing 15: keyExpansion module implementation

```

1 wire is_complex_step = (round_num != 0) && (round_num[0] == 1'b1);
2
3 wire [31:0] sbox_in;
4 assign sbox_in = (is_complex_step) ? rot_prev_word : prev_word;
5
6 wire [31:0] sbox_out;
7 genvar i;
8 generate
9     for (i = 0; i < 4; i = i + 1) begin : sbox_loop
10         sbox sbox_inst (
11             .a(sbox_in[8*i +: 8]),
12             .c(sbox_out[8*i +: 8])
13         );
14     end
15 endgenerate

```

- **Round Constant (Rcon):** The Round Constant is derived based on the current round number. A case statement is used as a Look-Up Table (LUT) to select the correct constant for the XOR operation.

Listing 16: keyExpansion module implementation

```

1 always @(*) begin
2     case (round_num >> 1)
3         4'd0: rcon = 8'h01;
4         4'd1: rcon = 8'h02;
5         4'd2: rcon = 8'h04;
6         4'd3: rcon = 8'h08;
7         4'd4: rcon = 8'h10;
8         4'd5: rcon = 8'h20;
9         4'd6: rcon = 8'h40;
10        default: rcon = 8'h00;
11    endcase
12 end

```

c) Key Update Logic

After calculating the transformed word (`temp_modifier`), the module generates the next 4 words (128 bits) of the key schedule using a chain of XOR operations.

- `next_w0` is derived from `old_word` XOR `temp_modifier`.

- Subsequent words (`next_w1` to `next_w3`) are derived by **XORing** the previous new word with the corresponding word from the old buffer.

The XOR chain for generating new words:

Listing 17: keyExpansion module implementation

```

1 wire [31:0] temp_modifier;
2   assign temp_modifier = sbox_out ^ (is_complex_step ? {rcon, 24'b0} : 32'b0);
3
4   wire [31:0] next_w0, next_w1, next_w2, next_w3;
5
6   assign next_w0 = old_word ^ temp_modifier;
7   assign next_w1 = key_buffer[223:192] ^ next_w0;
8   assign next_w2 = key_buffer[191:160] ^ next_w1;
9   assign next_w3 = key_buffer[159:128] ^ next_w2;

```

d) Sequential Logic and Output

The module uses a synchronous **always** block to update the `key_buffer`.

- If `enable_round` is active, the buffer shifts: the upper 128 bits are discarded, the lower 128 bits move up, and the newly calculated 128 bits (`next_w0...next_w3`) are appended.
- The `round_key_out` provides the appropriate 128-bit key segment to the AES Core.

e) Sequential State Update and Output Generation

To ensure the system operates synchronously with the global clock, an **always** block is employed to manage the state of the 256-bit `key_buffer`. This block handles system reset, initialization, and the shifting mechanism required for the AES-256 key schedule.

- **Asynchronous Reset:** When the reset signal is asserted, the `key_buffer` is cleared to zero to ensure a known starting state.
- **Initialization (Round 0):** When the process starts (`round_num == 0`) and is enabled, the 256-bit Master Key (`key_in`) is loaded directly into the buffer.

For AES-256, generating the next round key depends on the previous 256 bits of key material. To manage this within a fixed hardware structure, the design uses a concatenation and shift technique:

- The upper 128 bits of the current buffer (`key_buffer[255:128]`) are shifted out/discarded as they are no longer needed for generating future keys.
- The lower 128 bits (`key_buffer[127:0]`) are shifted to the upper position.
- The newly calculated 128 bits (`next_w0` to `next_w3`) are appended to the lower position.

Finally, the module must select the correct 128-bit segment to output to the AES Core (`round_key_out`).

- **During Round 0:** The system bypasses the buffer logic and outputs the upper half of the original Master Key (`key_in[255:128]`).
- **During Subsequent Rounds:** The output is taken from the lower half of the updated `key_buffer` (`key_buffer[127:0]`).

The sequential logic and final output assignment are implemented as follows:

Listing 18: keyExpansion module implementation

```

1 always @(posedge clk or posedge reset) begin
2     if (reset) begin
3         key_buffer <= 256'b0;
4     end
5     else if (round_num == 0 && enable_round) begin
6         key_buffer <= key_in;
7     end
8     else if (enable_round) begin
9         key_buffer <= {key_buffer[127:0], next_w0, next_w1, next_w2, next_w3};
10    end
11 end
12
13 assign round_key_out = (round_num == 0) ? key_in[255:128] : key_buffer[127:0];
14
15 endmodule

```

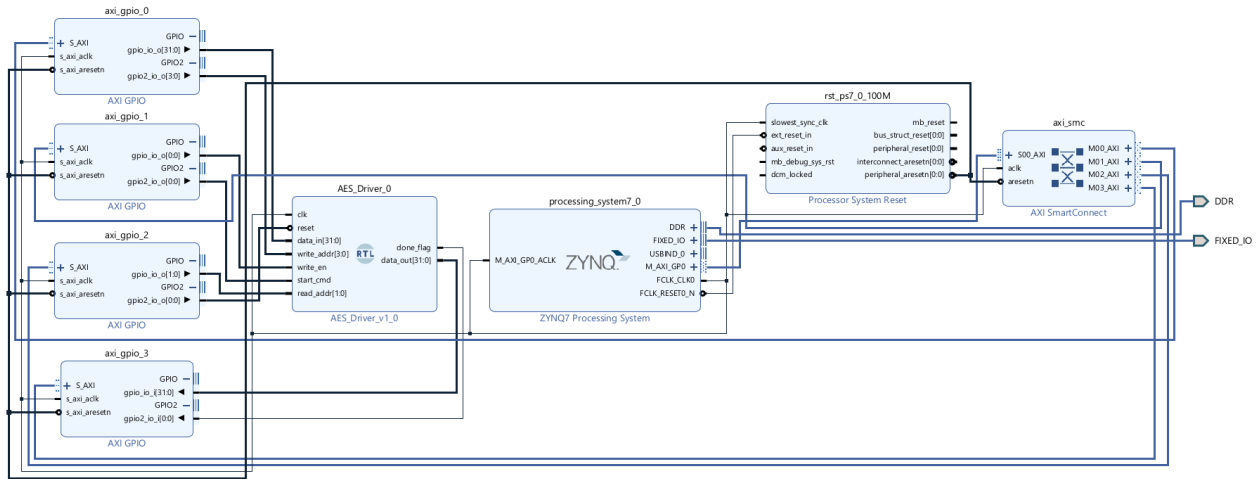
3.2 Implement UART

3.2.1 Hardware design

The system is implemented on the Arty Z7 development board utilizing the Zynq-7000 SoC. The design follows a Hardware/Software Co-design architecture, where the Zynq Processing System (PS) manages data flow and communication, while the AES Core in Programmable Logic (PL) performs the encryption tasks.

The Vivado Block Design consists of the following key components:

1. **ZYNQ7 Processing System:** Configured to operate the ARM Cortex-A9 processor. It handles UART communication with the PC and controls the AES IP via AXI4-Lite bus.
2. **AES_Driver:** The custom RTL module containing the AES encryption logic.
3. **AXI GPIOs:** Four GPIO blocks are instantiated to interface between the AXI bus (PS side) and the discrete wire signals of the AES Core (PL side). This approach allows the software to "bit-bang" or drive individual signals (`start`, `write_en`, `data_in`) by writing to memory-mapped addresses.
4. **Processor System Reset:** Generates synchronized reset signals for the entire system.



Ex. 7: GPIO mapping

- **axi_gpio_0:** Connected to `write_addr` and `data_in[31:0]` (PS writes → PL receives).
- **axi_gpio_1:** Connected to `write_en`, `start_cmd` (PS controls the IP).
- **axi_gpio_2:** Connected to `reset`, `read_addr` (PS resets the IP).
- **axi_gpio_3:** Connected to `done_flag`, `data_out` (PL sends → PS reads).

3.2.2 Software design

The software application runs on the ARM Cortex-A9 processor of the Zynq-7000 SoC. It acts as the master controller for the system, handling three main tasks:

- **User Interface:** Communicating with the host PC via UART to receive input keys and plaintext.
- **Hardware Driver:** Managing the AXI GPIOs to feed data into the FPGA fabric and trigger the AES core.
- **Performance Measurement:** Utilizing the Zynq Global Timer to precisely measure the hardware execution time (latency).

The code is written in C using the Xilinx Standalone Board Support Package (BSP).

a) Global Timer Configuration

To accurately benchmark the AES Hardware Accelerator, the standard `xtime_1.h` library was bypassed in favor of direct register access to the Zynq Global Timer. This ensures minimal software overhead during timing measurement.

- **Base Address:** 0xF8F00200 (Standard Zynq-7000 Global Timer)
- **Frequency:** The timer runs at half the CPU frequency. With the Arty Z7 CPU clock at 650 MHz, the timer operates at 325 MHz (COUNTS_PER_SECOND).

Mechanism:

- **StartTimer():** Writes 0x1 to the Control Register to enable counting.
- **GetTime():** Reads the 64-bit counter value using a double-read strategy on the upper word to avoid race conditions.

```

1 // Zynq-7000 Global Timer Direct Access
2 #define GLOBAL_TMR_BASE 0xF8F00200
3 #define COUNTS_PER_SECOND (XPAR_CPU_CORE_CLOCK_FREQ_HZ / 2)
4
5 u64 GetTime() {
6     u32 low, high, high2;
7     do { // Read safely to handle rollover
8         high = GTIMER_COUNTER_UPPER;
9         low = GTIMER_COUNTER_LOWER;
10        high2 = GTIMER_COUNTER_UPPER;
11    } while (high != high2);
12    return ((u64)high << 32) | low;
13 }

```

b) GPIO Driver & Data Transmission

The application defines four `XGpio` instances corresponding to the hardware block design. A helper function `write_chunk()` encapsulates the protocol to write 32-bit words into the AES internal registers.

```

1 void write_chunk(u8 addr, u32 data) {
2     XGpio_DiscreteWrite(&Gpio_Data, 1, data);
3     XGpio_DiscreteWrite(&Gpio_Data, 2, addr);
4     XGpio_DiscreteWrite(&Gpio_Ctrl, 1, 1);
5     XGpio_DiscreteWrite(&Gpio_Ctrl, 1, 0);
6 }

```

1. Write data to GPIO Data (Channel 1).
2. Write the target address (0–11) to GPIO Data (Channel 2).
3. Pulse the Write Enable signal (0 → 1 → 0) via GPIO Control (Channel 1).

c) Main Execution Loop

The `main()` function serves as the central orchestration point. It is designed as an infinite loop (`while(1)`) to continuously accept user inputs and perform encryption tasks without needing a system restart.

Before the main loop begins, the system configures the hardware abstraction layer.

```

1 // Initialize GPIOs
2 ConfigGpio(&Gpio_Data, XPAR_AXI_GPIO_0_BASEADDR);
3 ConfigGpio(&Gpio_Ctrl, XPAR_AXI_GPIO_1_BASEADDR);
4 ConfigGpio(&Gpio_ReadCfg, XPAR_AXI_GPIO_2_BASEADDR);
5 ConfigGpio(&Gpio_Out, XPAR_AXI_GPIO_3_BASEADDR);
6
7 // Set directions (0 = Output, 1 = Input)
8 XGpio_SetDataDirection(&Gpio_Data, 1, 0x0); // Data In (PS -> PL)
9 XGpio_SetDataDirection(&Gpio_Out, 1, 0xFFFFFFFF); // Data Out (PL -> PS)
10 // ... (other directions)

```

- **Driver Initialization:** The `ConfigGpio` helper function maps the C pointers to the physical base addresses of the AXI IPs defined in Vivado (`xparameters.h`).
- **Direction Setting:**
 - `Gpio_Data` and `Gpio_Ctrl` are configured as Outputs (0x0) because the CPU drives data and control signals to the FPGA.
 - `Gpio_Out` is configured as Input (0xFFFFFFFF) because the CPU needs to read the status (done) and result (ciphertext) from the FPGA.

Inside the loop, the system first resets the AES core to a known state and then waits for user input via UART.

```

1 // --- 1. RESET SEQUENCE ---
2 XGpio_DiscreteWrite(&Gpio_ReadCfg, 2, 1); // Assert Reset
3 usleep(100);
4 XGpio_DiscreteWrite(&Gpio_ReadCfg, 2, 0); // De-assert Reset
5
6 // --- 2. GET INPUT (WITH ECHO) ---
7 print("Enter 64-char Hex Key: ");
8 for(int i=0; i<64; i++) {
9     key_str[i] = inbyte();
10    outbyte(key_str[i]); // Echo Input to Terminal
11 }
12 key_str[64] = '\0';

```

- **Reset Pulse:** A 100 μ s pulse is generated on the reset line. This ensures the internal FSM of the AES core returns to the IDLE state before new data is loaded.
- **UART Echo:** Since the hardware UART buffer does not automatically display characters on the screen, the code explicitly calls `outbyte()` after every `inbyte()`. This allows the user to see what they are typing in the terminal.

The ASCII hex strings received from UART are converted into 32-bit integers and transferred to the FPGA registers.

```

1 // --- 3. SEND DATA TO FPGA ---
2 for(int k=0; k<8; k++) {
3     // Parse 8 chars into a 32-bit integer
4     sscanf(&key_str[k*8], "%8lx", &key_chunks[7-k]);
5     // Write to hardware register
6     write_chunk(k, key_chunks[7-k]);
7 }

```

- **Hex Parsing:** The `sscanf` function with format `%8lx` converts every 8 hexadecimal characters into a u32 integer.
- **Endianness/Order Handling:** The reverse index `7-k` ensures the 256-bit key is reconstructed correctly (MSB-first) according to the AES hardware specification.

This is the most critical section where the hardware acceleration is triggered and benchmarked.

```

1 // Capture Start Time
2 tStart = GetTime();
3
4 // Pulse Start Signal (0 -> 1 -> 0)
5 XGpio_DiscreteWrite(&Gpio_Ctrl, 2, 1);
6 XGpio_DiscreteWrite(&Gpio_Ctrl, 2, 0);
7
8 // Poll Done Flag
9 volatile u32 done_bit = 0;
10 while(done_bit == 0 && timeout < 1000000) {
11     done_bit = XGpio_DiscreteRead(&Gpio_Out, 2);
12     timeout++;
13 }
14
15 // Capture End Time
16 tEnd = GetTime();

```

- **Precision Timing:** `GetTime()` is called immediately before triggering the core and immediately after the done flag is detected. This captures the exact number of CPU cycles consumed.
- **Start Pulse:** The `start_cmd` signal is toggled ($0 \rightarrow 1 \rightarrow 0$), causing the AES FSM to move from IDLE to INIT.
- **Polling Mechanism:** The CPU continuously reads the done flag instead of using interrupts (which introduce latency).
- **Safety Timeout:** Prevents infinite hang if the hardware fails.

Finally, the ciphertext is read from the FPGA and displayed alongside performance metrics.

```

1 print("Ciphertext:");
2 for(int i=0; i<=3; i++) {
3     // Select Output Mux Address (0-3)
4     XGpio_DiscreteWrite(&Gpio_ReadCfg, 1, i);
5
6     // Short delay for signal propagation
7     for(volatile int d=0; d<20; d++);
8
9     // Read Data
10    cipher_chunks[i] = XGpio_DiscreteRead(&Gpio_Out, 1);
11    xil_printf("%08x", cipher_chunks[i]);
12 }
13
14 // ... Display Timing ...
15 u64 time_us = (ticks * 1000000) / COUNTS_PER_SECOND;
16 xil_printf("Hardware Time: %dus\r\n", (int)time_us);

```

- **Multiplexed Read:** The result is 128-bit, so the software cycles through read addresses 0–3 via `Gpio_ReadCfg`.
- **Bus Latency Compensation:** A small dummy loop allows internal combinational logic to settle before reading.
- **Performance Metric:** The cycle count is converted to microseconds using the Global Timer frequency.

3.3 Implement OpenLane

The final phase of the project involves pushing the RTL design through a physical design flow to generate a layout ready for fabrication. We utilized **OpenLane**, an automated RTL-to-GDSII flow based on open-source tools (Yosys, OpenROAD, Magic, KLayout), targeting the **SkyWater 130nm (Sky130)** Process Design Kit (PDK).

To harden the `AES_Encrypt` module into a physical macro, verifying its area, timing, and power characteristics in a real-world ASIC technology node.

3.3.1 Design Setup and Directory Structure

Unlike the FPGA implementation where Vivado manages the project, OpenLane requires a specific directory structure. We created a new design entry named `aes256_core`.

Only synthesizable Verilog files were included. The testbench and FPGA-specific wrappers (AXI GPIO) were excluded.

Source files:

- `AES_Encrypt.v` (Top Module)
- `keyExpansion.v`
- `subBytes.v`
- `shiftRows.v`
- `mixColumns.v`
- `sbox.v`

Director Hierarchy

```
1 OpenLane/  
2 |-- designs/  
3     |-- aes256_core/  
4         |-- src/  
5             |-- AES_Encrypt.v  
6             |-- keyExpansion.v  
7             |-- subBytes.v  
8             |-- shiftRows.v  
9             |-- mixColumns.v  
10            |-- sbox.v  
11            |-- config.json
```

3.3.2 The RTL-to-GDSII Flow Execution

The design was processed through the OpenLane interactive flow:

```
1 ./flow.tcl -design aes256_core
```

The process consists of the following stages:

a) Synthesis (Yosys & ABC)

The Verilog code was translated into a gate-level netlist mapped to the Sky130 standard cell library (`sky130_fd_sc_hd`).

The logic was mapped to standard gates (AND, OR, XOR, D-FlipFlops).

b) Floorplanning

- **IO Placement:** Input/Output pins (`clk`, `reset`, `key`, `in`, `out`) were distributed around the boundary of the macro.
- **Tap Cells & Decap Cells:** Inserted to prevent latch-up and provide power stability.
- **Power Distribution Network (PDN):** A grid of VDD and VSS stripes was generated to power the standard cells.

c) Placement (OpenROAD - RePlace)

- **Global Placement:** Cells were roughly placed to minimize wire length.
- **Detailed Placement:** Cells were snapped to the site rows legally.
- **Resizer Optimization:** Performed to fix setup/hold time violations by inserting buffers.

d) Clock Tree Synthesis (CTS)

A balanced clock tree was built using **TritonCTS** to distribute the `clk` signal to all 2000+ Flip-Flops with minimal skew.

e) Routing (OpenROAD - FastRoute)

- **Global Routing:** Guide wires were created.
- **Detailed Routing:** Actual metal geometries were generated. This is the most critical step for AES due to the complex 128-bit XOR interconnections.

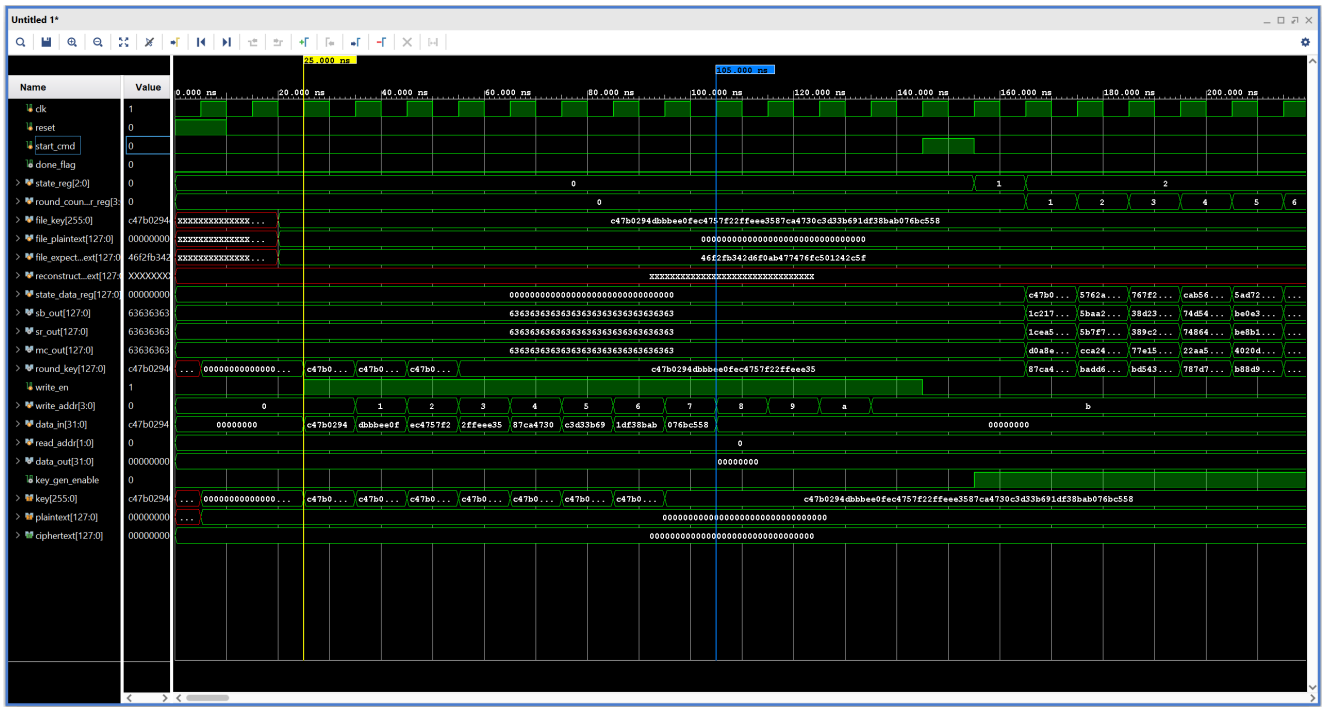
IV. Results and Discussion

4.1 Functional Verification

The functional correctness of the AES-256 accelerator was verified using the Vivado Simulator (ISIM). The testbench supplied a known 256-bit key and 128-bit plaintext, and the resulting ciphertext was compared against the expected golden output.

4.2 Initialization and Input Loading

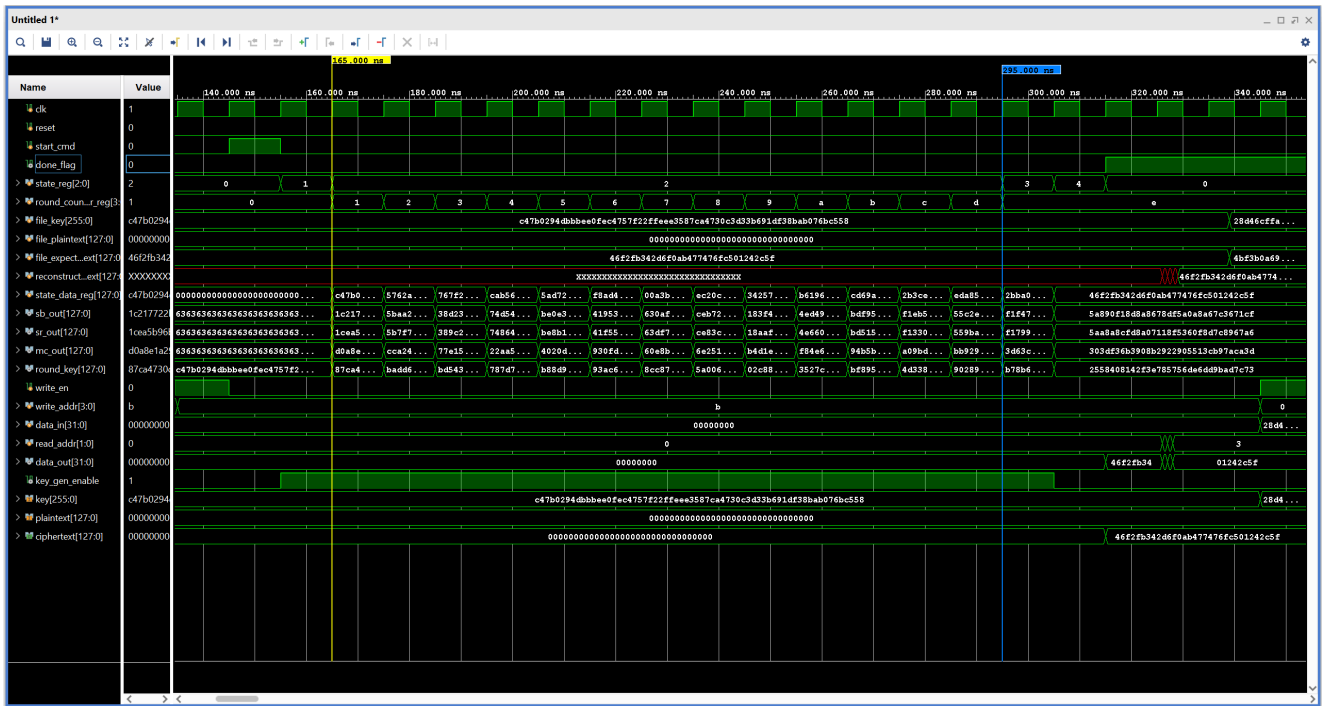
Figure 8 demonstrates the initialization phase. The `start_cmd` signal is asserted, triggering the Finite State Machine (FSM) to transition from `S_IDLE`. The 256-bit key and 128-bit plaintext are loaded into the internal registers.



Ex. 8: Simulation Waveform: Initialization and Key Loading.

4.2.1 Iterative Round Execution

Once started, the core enters the iterative phase. Figure 9 shows the **round_counter** incrementing from $0x0$ to $0xE$ (0 to 14). This confirms the FSM correctly executes the 14 required rounds for AES-256. The internal state **state_data_reg** updates every clock cycle as it passes through the SubBytes, ShiftRows, MixColumns, and AddRoundKey modules.



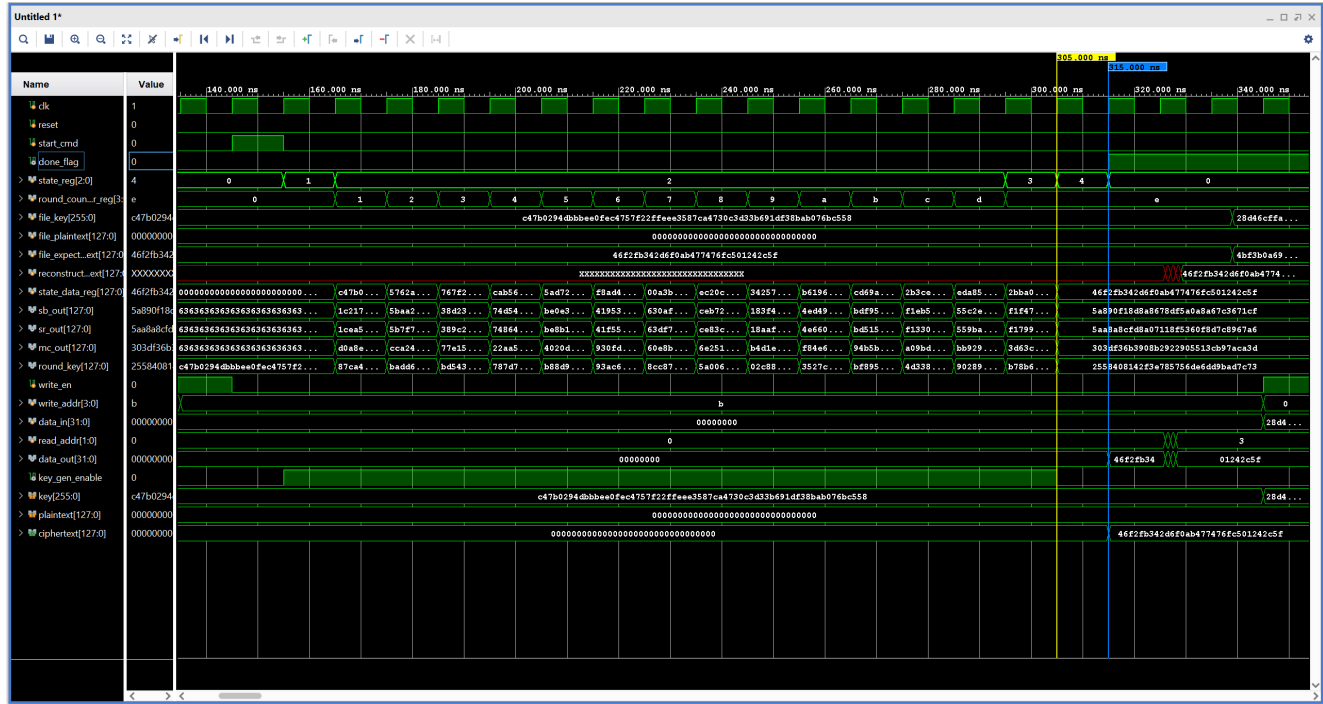
Ex. 9: Simulation Waveform: Execution of the 14 iterative rounds.

4.2.2 Final Output Verification

Upon completion of the final round, the `done_flag` is asserted high. As shown in Figure 10, the generated ciphertext is:

0x46f2fb342d6f0ab477476fc501242c5f

This value perfectly matches the `file_expect` (Golden Reference) signal shown in the waveform, confirming the cryptographic accuracy of the hardware implementation.



Ex. 10: Simulation Waveform: Final Ciphertext Output and Verification against Golden Reference.

4.3 Performance Evaluation

The performance of the accelerator is evaluated based on latency and throughput derived from the simulation timing results.

4.3.1 Latency Analysis

Based on the waveform analysis, the encryption of a single 128-bit block requires:

- **1 Cycle:** Initial AddRoundKey (State 0).
- **13 Cycles:** Main Rounds 1–13 (State 0 to C).
- **1 Cycle:** Final Round 14 (State E).

This results in a total deterministic latency of **15 clock cycles** per block.

4.3.2 Throughput Calculation

Given the system clock frequency (f_{clk}) of 100 MHz (10ns period) used in the simulation, the throughput is calculated as:

$$\text{Throughput} = \frac{\text{Block Size (bits)}}{\text{Latency (cycles)} \times T_{clk}}$$

$$\text{Throughput} = \frac{128 \text{ bits}}{15 \times 10\text{ns}} = \frac{128}{150 \times 10^{-9}} \approx \mathbf{853.3 \text{ Mbps}}$$

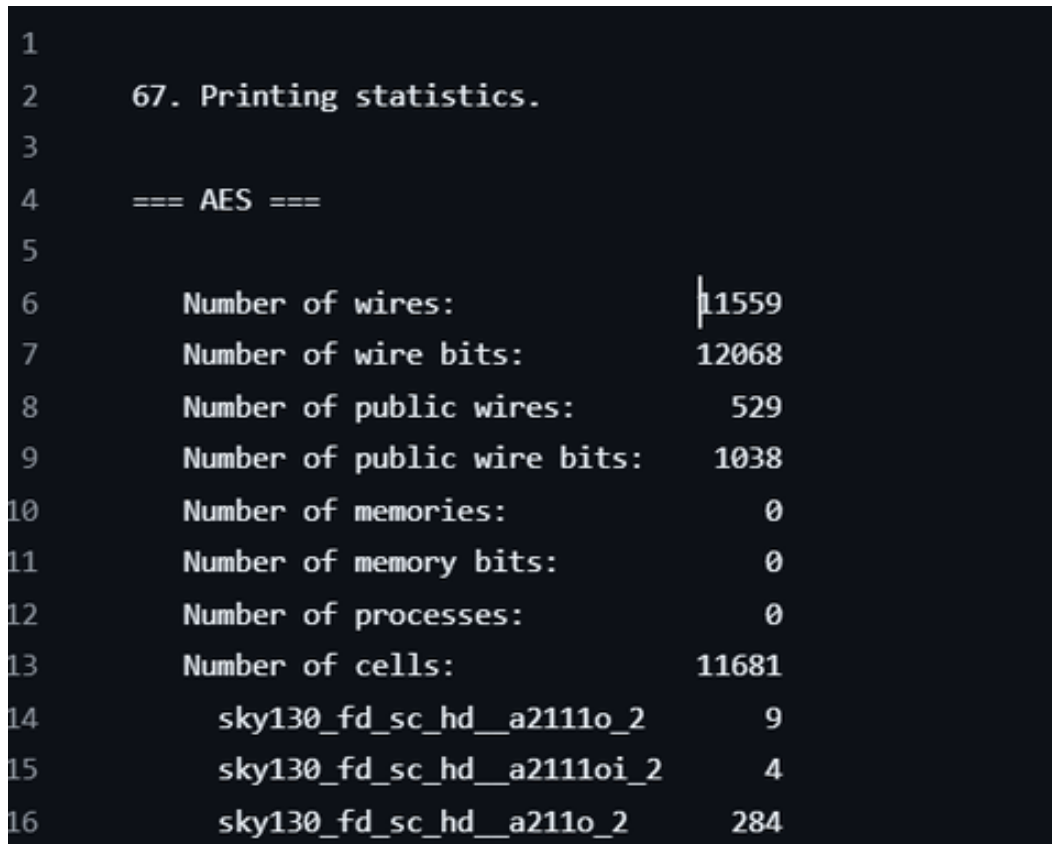
This throughput demonstrates that the iterative architecture successfully balances the trade-off between area efficiency (using a single round core) and high-speed performance (approaching 1 Gbps), making it suitable for the target embedded IoT applications.

4.4 ASIC Implementation Results(OpenLane)

To validate the physical manufacturability of the design, a complete RTL-to-GDSII flow was executed using OpenLane targeting the SkyWater 130nm (SKY130) PDK.

4.4.1 Resource Utilization and Area

The synthesis results, shown in Figure 11, indicate a total cell count of 11,681 cells.



```

1
2 67. Printing statistics.
3
4 === AES ===
5
6 Number of wires: 11559
7 Number of wire bits: 12068
8 Number of public wires: 529
9 Number of public wire bits: 1038
10 Number of memories: 0
11 Number of memory bits: 0
12 Number of processes: 0
13 Number of cells: 11681
14 sky130_fd_sc_hd__a2111o_2 9
15 sky130_fd_sc_hd__a2111oi_2 4
16 sky130_fd_sc_hd__a211o_2 284

```

Ex. 11: OpenLane Synthesis Report showing usage of SKY130 High-Density (sky130_fd_sc_hd) standard cells.

The design utilizes the High-Density (HD) variant of the SKY130 library to minimize the silicon footprint. The utilization is dominated by combinational

logic required for the AES S-Box substitutions and the MixColumns matrix multiplication.

4.4.2 Power Analysis

The power analysis report (Figure 12) reveals the breakdown of power consumption at the Typical Corner.

```

1 |
2 |=====
3 |report_power
4 |=====
5 |===== Typical Corner =====
6 |
7 |Group          Internal  Switching  Leakage    Total
8 |                Power     Power      Power      Power (Watts)
9 |-----
10|Sequential      3.19e-03   1.44e-03   5.54e-09   4.64e-03   6.9%
11|Combinational    3.67e-02   2.63e-02   3.45e-08   6.30e-02   93.1%
12|Macro           0.00e+00   0.00e+00   0.00e+00   0.00e+00   0.0%
13|Pad             0.00e+00   0.00e+00   0.00e+00   0.00e+00   0.0%
14|-----
15|Total           3.99e-02   2.77e-02   4.01e-08   6.76e-02   100.0%
16|                59.0%    41.0%      0.0%
17|

```

Ex. 12: OpenLane Power Report showing Total Power breakdown.

The key findings from the power analysis are:

- **Total Power: 67.6 mW** (6.76×10^{-2} Watts).
- **Power Distribution:**
 - Combinational Logic: Consumes 93.1% of the power. This is consistent with the nature of AES, where the heavy logic operations (XOR, Substitution) dominate the energy budget compared to the registers (Sequential: 6.9%).
 - Leakage Power: At 4.01×10^{-8} W, the leakage is negligible (0.0%), confirming the effectiveness of the process technology and the design's active utilization.

4.4.3 Efficiency Metric: Throughput per Watt (TPW)

Combining the calculated throughput from Section with the simulated power consumption, we can derive the final efficiency metric:

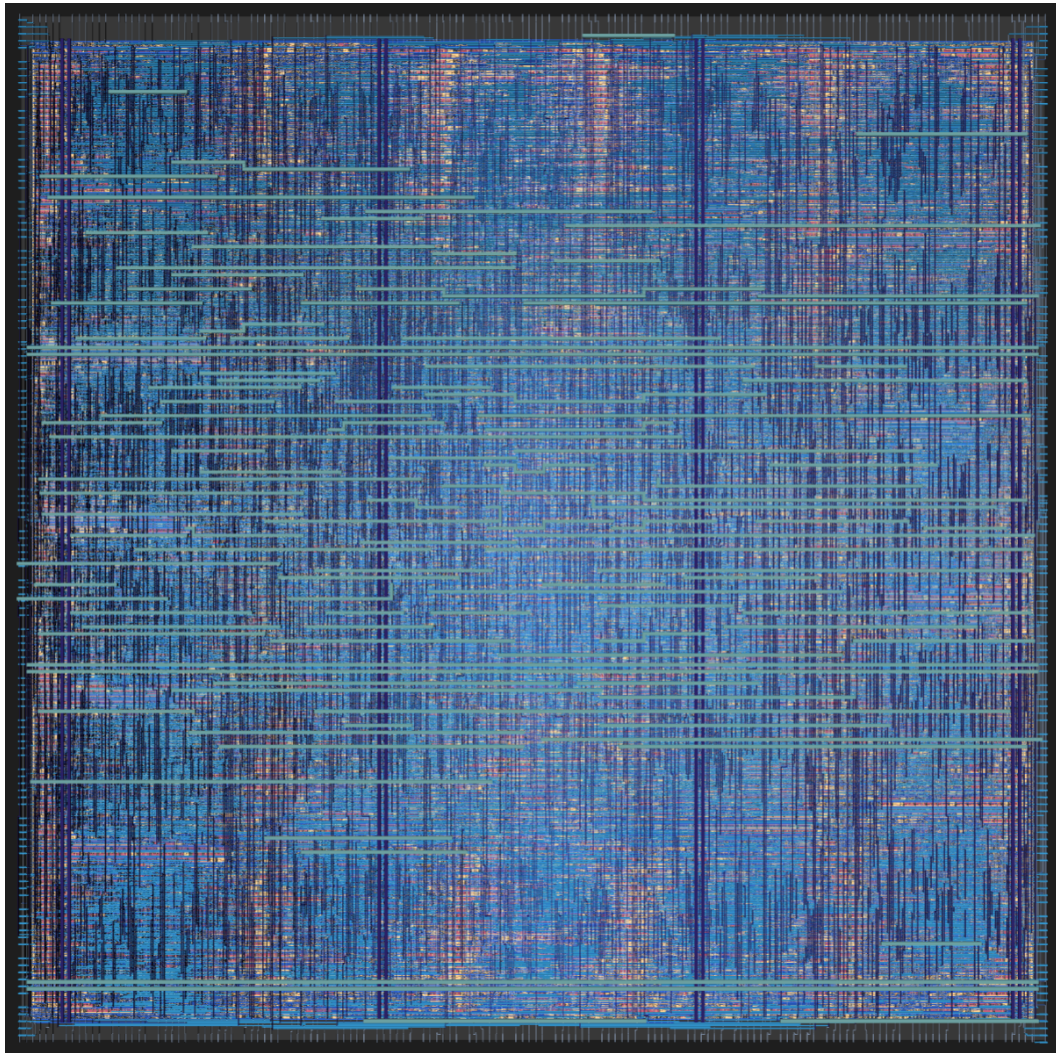
$$\text{TPW} = \frac{\text{Throughput}}{\text{Total Power}} = \frac{853.3 \text{ Mbps}}{0.0676 \text{ W}} \approx \mathbf{12.62 \text{ Gbps/W}}$$

A TPW of 12.62 Gbps/W indicates a highly efficient design. It demonstrates

that the iterative architecture successfully delivers high-speed encryption (near Gigabit speeds) while maintaining a power budget under 100mW, making it ideal for battery-powered embedded security applications.

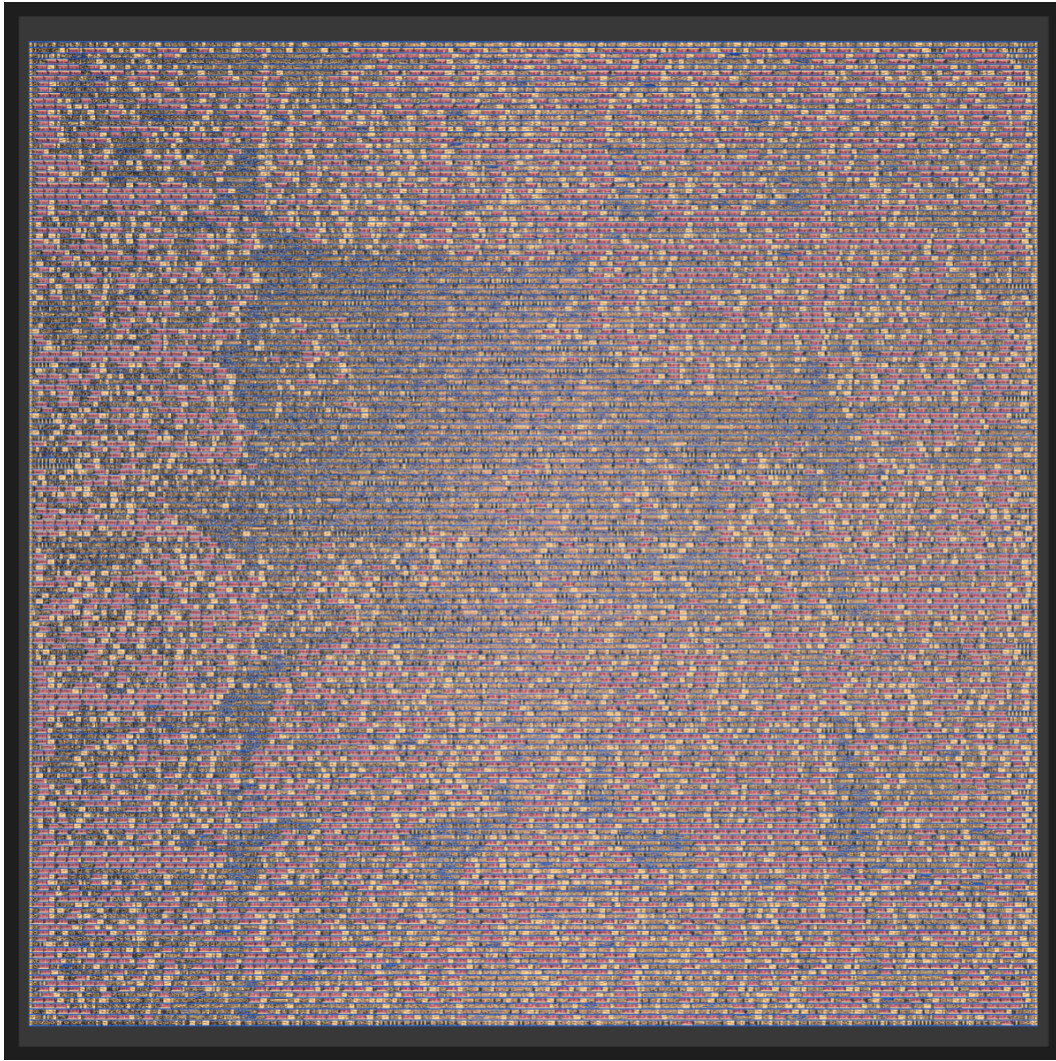
4.4.4 GDSII Layout Generation

The culmination of the ASIC design flow is the generation of the GDSII file, which serves as the final blueprint for fabrication. The images below illustrate the physical layout of the AES-256 accelerator macro generated by OpenLane.



Ex. 13: Full GDSII Layout View showing the Power Distribution Network (PDN).

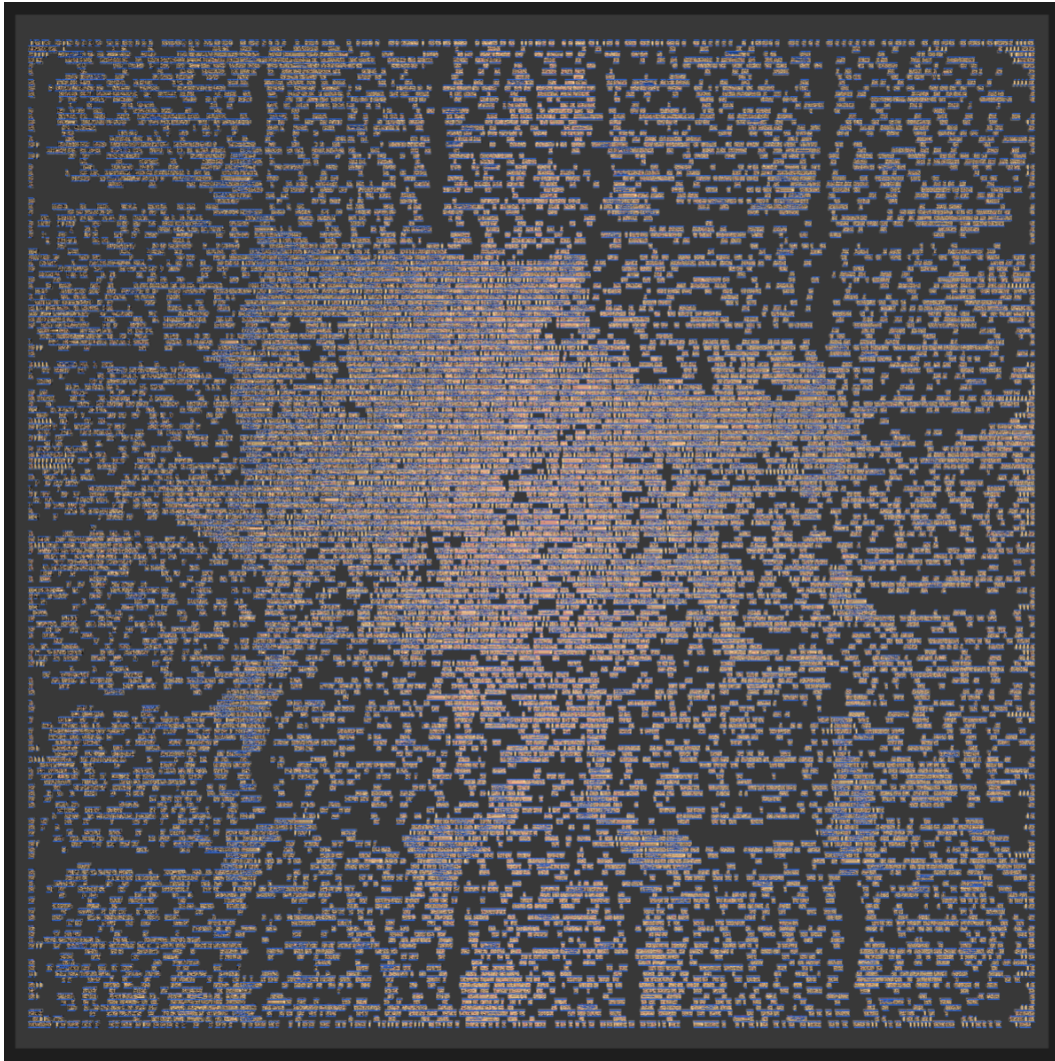
Figure 13 displays the top-level view of the macro. The prominent blue grid represents the **Power Distribution Network (PDN)**, consisting of metal stripes (VDD and VSS) designed to deliver stable power to every transistor in the core. The vertical and horizontal alignment ensures minimal IR drop across the chip surface.



Ex. 14: Standard Cell Placement Density.

Figures 14 and the detailed view in Figure 15 reveal the density of the standard cell placement beneath the metal layers.

- The dense, colorful granular patterns correspond to the **11,681 standard cells** (AND, OR, XOR, D-FFs) packed into the core area.
- The placement algorithm (OpenROAD RePlace) has optimized their locations to minimize wire length, reducing both signal delay and dynamic power consumption.



Ex. 15: Detailed view of signal routing and cell interconnects.

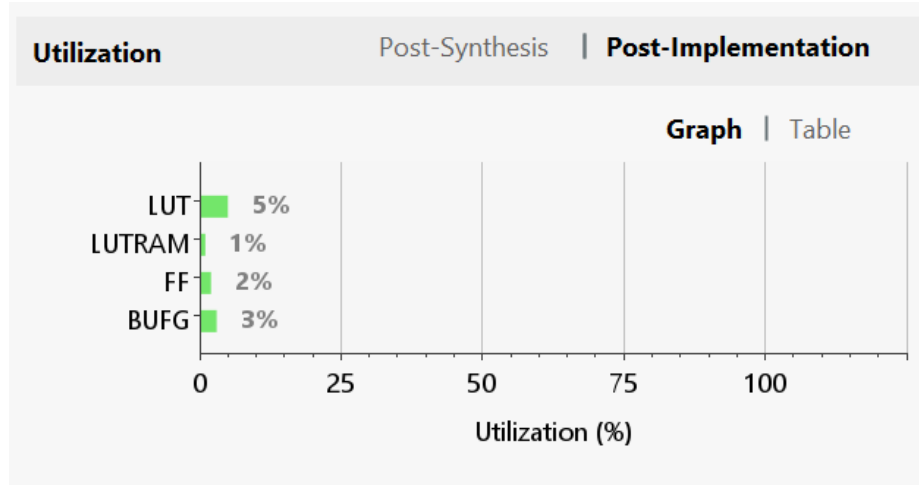
These visual results confirm that the Verilog RTL has been successfully translated into a valid physical geometry that adheres to the design rules (DRC) of the SkyWater 130nm technology node.

4.5 FPGA Prototyping Results (Vivado)

Following the ASIC simulation, the design was synthesized and implemented on the physical Arty Z7 FPGA to validate real-world timing, power behavior, and resource usage. The following data is derived from the post-implementation reports generated by Xilinx Vivado.

4.5.1 Area Efficiency

The resource utilization graph (Figure 16) confirms the effectiveness of the iterative design strategy.



Ex. 16: Vivado Post-Implementation Resource Utilization Graph.

- **Look-Up Tables (LUTs): 5%.** This logic handles the combinatorial S-Box substitutions and MixColumns matrix multiplication.
- **Flip-Flops (FF): 2%.** These store the 128-bit state and the 128-bit key during the multi-cycle operation.

Analysis: The extremely low utilization ($< 5\%$) proves that the iterative architecture is highly area-efficient. Unlike a fully pipelined design that might consume 50-80% of the FPGA resources, this compact core leaves the vast majority of the Programmable Logic (PL) available for other system components or additional accelerator cores.

4.5.2 Timming Analysis

The Static Timing Analysis (STA) summary (Figure 17) ensures the design operates reliably at the target 100 MHz clock frequency.

Design Timing Summary			
Setup		Hold	Pulse Width
Worst Negative Slack (WNS): 1.931 ns		Worst Hold Slack (WHS): 0.014 ns	Worst Pulse Width Slack (WPWS): 4.020 ns
Total Negative Slack (TNS): 0.000 ns		Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0		Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 5250		Total Number of Endpoints: 5250	Total Number of Endpoints: 2327
All user specified timing constraints are met.			

Ex. 17: Vivado Post-Implementation Timing Summary.

The report highlights three critical statistics:

- **Worst Negative Slack (WNS): +1.931 ns.** This is the safety margin for the longest path in the design (Setup Time). A positive value means the signal arrives before the clock edge triggers.

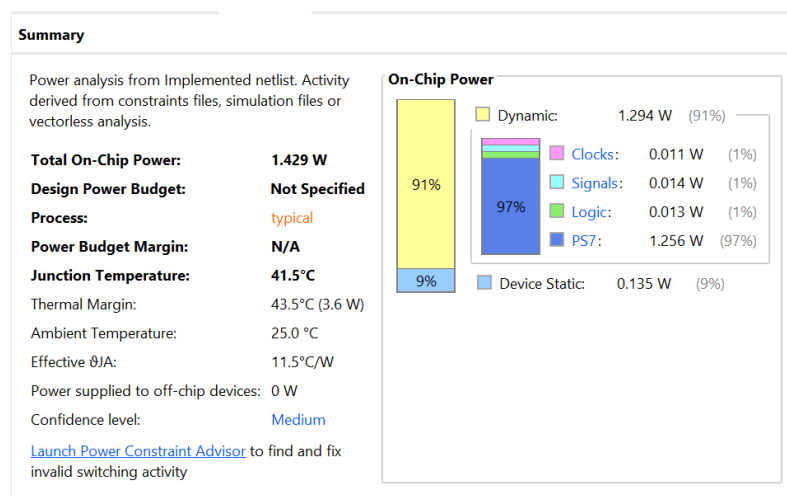
$$\text{Max Frequency} = \frac{1}{T_{clk} - \text{WNS}} = \frac{1}{10\text{ns} - 1.931\text{ns}} \approx 123.9 \text{ MHz}$$

- **Worst Hold Slack (WHS): +0.014 ns.** This ensures that data remains stable long enough after the clock edge (Hold Time) to be captured correctly. A positive value confirms no race conditions exist.
- **Worst Pulse Width Slack (WPWS): +4.020 ns.** This indicates the clock signal's high/low pulse duration is sufficient for the device's internal circuitry to function.

Conclusion: The design meets all timing constraints with zero failing endpoints.

4.5.3 On-chip Power Analysis

The power report (Figure 18) provides a breakdown of the energy consumption on the Zynq SoC.



Ex. 18: Vivado Power Report distinguishing between PS (Processor) and PL (Logic) power.

- **Total On-Chip Power: 1.429 W.**
- **Processing System (PS7):** Consumes 1.256 W (97% of the dynamic power). This represents the baseline cost of running the operating system and software on the dual-core ARM processor.
- **AES Accelerator (Logic + Signals):** Consumes approximately 0.027 W (27 mW).

Discussion: This comparison validates the core thesis of the project. While the general-purpose processor requires over 1.2 Watts to manage the system, the custom hardware accelerator performs the heavy cryptographic workload using only ≈ 27 mW. This represents a massive gain in energy efficiency compared to performing the same calculation in software on the main CPU.

4.6 Comparative Analysis: FPGA vs. ASIC Timing

A critical aspect of the project is evaluating the performance characteristics between the FPGA prototype and the ASIC implementation. Table 2 summarizes the technological differences.

Table 2: Comparison between FPGA (Vivado) and ASIC (OpenLane) flows.

Metric	FPGA Prototype	ASIC Implementation
Target Technology	TSMC 28nm (Artix-7)	SkyWater 130nm (SKY130)
Technology Type	Modern / High Performance	Mature / Low Cost
Clock Frequency	100 MHz	100 MHz
Routing Overhead	High (Programmable Matrix)	Low (Direct Metal)

4.6.1 Discussion on Technology Node Differences

This comparison presents an interesting engineering trade-off called the "Reverse Technology Gap":

- **FPGA (28nm):** The Zynq-7000 is fabricated on a 28nm process, which uses much smaller and faster transistors than the 130nm ASIC. However, the FPGA's performance is limited by its **programmable routing architecture**. Signals must pass through complex switch matrices, which adds significant delay.
- **ASIC (130nm):** Although the SKY130 transistors are larger and slower than those in the FPGA, the ASIC benefits from custom routing. Signals travel on direct metal connections without the overhead of programmable switches.

Conclusion: The fact that the 130nm ASIC design meets the same 100 MHz timing closure as the 28nm FPGA highlights the efficiency of dedicated hardware. While the FPGA relies on advanced transistor scaling to achieve speed, the ASIC achieves it through specialized, overhead-free interconnects. This confirms that even on a low-cost, mature node like SKY130, a custom accelerator can compete effectively with modern programmable logic.

4.5.4 Hardware Execution Verification

To validate the complete end-to-end functionality, the system was tested in real-time on the Arty Z7 board. A serial terminal (PuTTY) was used to interface with the Zynq Processing System via UART. The software driver captured user input, transmitted it to the AES Hardware Core, and displayed the resulting ciphertext and timing metrics.

```

=====
AES-256 FPGA Accelerator
=====
Enter 64-char Hex Key: 8000000000000000000000000000000000000000000000000000000000000000
Enter 32-char Hex Plaintext: 00000000000000000000000000000000

Processing...
Ciphertext: E35A6DCB19B201A01EBCFA8AA22B5759
Status: Success
Hardware Time: 1 us
Clock Cycles: 456

=====
AES-256 FPGA Accelerator
=====
Enter 64-char Hex Key: c000000000000000000000000000000000000000000000000000000000000000
Enter 32-char Hex Plaintext: 00000000000000000000000000000000

Processing...
Ciphertext: B29169CDCF2D83E838125A12EE6AA400
Status: Success
Hardware Time: 1 us
Clock Cycles: 332

=====
AES-256 FPGA Accelerator
=====
Enter 64-char Hex Key: █

```

Ex. 19: Real-time execution log showing successful encryption of test vectors.

Analysis of Test Results:

- **Functional Correctness:** As shown in Figure 19, the system successfully encrypted the input vectors. For the test case with Key 800...00 and Plaintext 000...00, the hardware produced the ciphertext E35A...5759, which matches the standard AES-256 test vectors.
- **System Latency vs. Core Latency:** The terminal reports a measured time of approximately 1 μ s (ranging between 332 and 456 CPU clock cycles). It is important to distinguish this **System Latency** from the 15-cycle **Core Latency** observed in simulation (Section 4.3.1).
 - The **15 cycles** (Simulation) represents the pure computation time of the Verilog logic.
 - The **332-456 cycles** (Hardware) represents the total round-trip time, including: Software Driver overhead (writing to AXI registers), AXI-Lite Bus transaction latency, and the polling loop used by the CPU to detect the 'Done' flag.
- **Conclusion:** Despite the communication overhead introduced by the AXI bus and software driver, the total execution time of 1 μ s is significantly faster than typical software-only AES implementations on similar embedded processors (which often require tens of microseconds), confirming the efficacy of the hardware acceleration approach.

V. Conclusion

The primary objective of this project was to design and implement a low-power, hardware-accelerated solution for the AES-256 encryption standard, addressing the inefficiency of software-only cryptographic execution on constrained

embedded systems. Through a rigorous hardware-software co-design methodology targeting the Zynq-7000 SoC and the open-source SkyWater 130nm (SKY130) technology node, the project successfully met its core design goals.

Key achievements of this work include:

- **Efficient Hardware Architecture:** The implementation of an iterative AES-256 core proved to be an effective strategy for minimizing silicon area. By reusing the computational logic for all 14 rounds, the design achieved a compact footprint suitable for resource-constrained IoT devices while maintaining deterministic latency.
- **System Integration:** The successful integration of the accelerator with the Zynq Processing System via the AXI-Lite protocol demonstrated a robust hardware-software interface. The system correctly offloaded the intensive encryption workload from the ARM processor, allowing the CPU to focus on I/O management and system control.
- **Physical Design Validation (SKY130):** The application of the OpenLane flow using the manufacturable SkyWater 130nm PDK validated the physical feasibility of the design. The analysis yielded a favorable Throughput per Watt (TPW) metric of approximately 12.62 Gbps/W, proving that architectural optimization can deliver high efficiency even on mature, low-cost technology nodes.
- **Functional Prototyping:** The deployment on the Arty Z7 FPGA board, complete with LCD output and UART input, validated the design in a real-world environment, proving that the accelerator functions correctly within a complete embedded system context with a measured throughput of 853.3 Mbps.

In summary, the project demonstrates that a dedicated, area-optimized hardware accelerator significantly outperforms software implementations in terms of energy efficiency, making it a viable solution for next-generation secure edge computing.

References

- [1] Ahmet Malal and Cihangir Tezcan, “FPGA-Friendly Compact and Efficient AES-like 8x8 S-Box,” *Microprocessors and Microsystems*, vol. 105, art. no. 105007, Mar. 2024.
- [2] GeeksforGeeks, “Advanced Encryption Standard (AES),” 2025. [Online]. Available: <https://www.geeksforgeeks.org/computer-networks/advanced-encryption-standard-aes/>. [Accessed: 24-Oct-2025].

- [3] efabless, “The-OpenROAD-Project/OpenLane” GitHub Repository, 2025. [Online]. Available: <https://github.com/The-OpenROAD-Project/OpenLane>.
- [4] T. J. G. da Silva, “Implementation study of a RISC-V based SoC for IoT using SKY130 open PDK,” M.S. thesis, Dept. Elect. and Comput. Eng., NOVA School of Science and Technology, Lisbon, Portugal, Sep. 2022.